

الفهرس

2	 <i>Task 1: Data Acquisition and Preparation</i>
6	 <i>Task 2: LSTM Model Development</i>
11	 <i>Task 3: Model Evaluation and Prediction</i>
15	 <i>Task 4: Visualization Enhancement and Analysis</i>
19	 <i>Task 5: Fine-Tuning and Comparison</i>
22	 <i>Task 6: Future Price Forecast</i>
25	 <i>Task 8: Creative Visualization</i>

Task 1: Data Acquisition and Preparation

1. Use the `yfinance` library to retrieve historical gold price data from Yahoo Finance.

```
import yfinance as yf
import pandas as pd
from sklearn.model_selection import train_test_split

#1.1 Use the `yfinance` library to retrieve historical gold price data from Yahoo Finance.
# Define the ticker symbol for gold futures
ticker_symbol = 'GC=F'

# Download the historical data for gold
gold_data = yf.download(ticker_symbol, start='2020-01-01', end='2024-06-10')

# Display the first few rows of the data
print(gold_data.head())

# Optionally, save the data to a CSV file
gold_data.to_csv('historical_gold_prices.csv')
```

لاسترجاع بيانات سعر الذهب التاريخية
الخطوات:

اولا استيراد المكتبات:

بتحميل المكتبات اللازمة للعمل. تستخدم المكتبة yfinance لاسترجاع

البيانات من Yahoo Finance

وتستخدم المكتبة pandas للتعامل مع البيانات وتستخدم sklearn

لاستخدام وظيفة train_test_split

ثانيا تعريف رمز الأداة المالية:

تعريف متغير ticker_symbol بقيمة 'GC=F' , وهي رمز آجل الذهب

ثالثا تحميل البيانات التاريخية للذهب

استخدام الدالة yf.download() لاسترجاع البيانات التاريخية للذهب من

Yahoo Finance. يتم تحديد تاريخ البدء بـ '01-01-2020' وتاريخ

الانتهاء بـ '10-06-2024'. يتم تخزين البيانات في المتغير gold_data

رابعا عرض الصفوف الأولى من البيانات

استخدام الدالة print() لعرض الصفوف الأولى من البيانات المسترجعة.

يتم استدعاء الدالة head() على المتغير gold_data لعرض الصفوف

الأولى.

خامسا حفظ البيانات في ملف
حفظ البيانات في ملف CSV ، يمكن استخدام الدالة to_csv() لحفظ
المتغير gold_data في ملف بالاسم 'historical_gold_prices.csv'.

2. Preprocess the data, handling any missing values and formatting timestamps and prices.

```
gold_data= gold_data.dropna()

#formatting timestamps and prices
gold_data.reset_index(inplace=True)
gold_data = gold_data[['Date', 'Close']]
gold_data.columns = ['timestamp', 'price']

print(gold_data.head())
```

تنسيق البيانات المسترجعة والتأكد من خلوها من القيم المفقودة.
الخطوات:

اولا استبعاد القيم المفقودة:

استخدام الدالة dropna() لاستبعاد أي صفوف تحتوي على قيم مفقودة
من المتغير gold_data. هذا يساعد في التأكد من أن البيانات لا تحتوي
على أي قيم مفقودة.

ثانيا تنسيق الأختام الزمنية والأسعار:

استخدام الدالة reset_index() لإعادة تعيين فهرس الصفوف في المتغير
gold_data. بعد ذلك، يتم تحديد الأعمدة التي ستتم استخدامها،
وهي 'data' 'Close' ، باستخدام المربعات التي تحيط بالأعمدة
المطلوبة. ثم يتم تغيير أسماء الأعمدة إلى 'price' ' timestamp'
باستخدام خاصية columns

ثالثا عرض الصفوف الأولى من البيانات:

يستخدم الدالة print() لعرض الصفوف الأولى من المتغير gold_data
بعد إجراء التغييرات المذكورة أعلاه. يتم استدعاء الدالة head()
المتغير gold_data لعرض الصفوف الأولى بعد التنسيق.

3. Split the dataset into training and testing sets, ensuring the testing set covers the most recent data.

```
#1.3 Split the dataset into training and testing sets, ensuring the testing set covers the most recent data.
train_size = 0.8 # 80% training, 20% testing

train_data, test_data = train_test_split(gold_data, test_size=1 - train_size, shuffle=False)

# Display the shapes of training and testing sets
print("Training set shape:", train_data.shape)
print("Testing set shape:", test_data.shape)
```

[*****100%*****] 1 of 1 completed Open

Date

لتقسيم البيانات إلى مجموعة تدريب ومجموعة اختبار الخطوات:

أولاً تحديد حجم مجموعة التدريب:
تحدد المتغير `train\_size` بقيمة 0.8، وهذا يعني أن مجموعة التدريب ستشمل 80% من البيانات، بينما ستشمل مجموعة الاختبار 20% من البيانات.

ثانياً تقسيم البيانات:

استخدام الدالة `train\_test\_split` مع المتغير `gold\_data` كعامل الإدخال الأول. المعامل `test\_size` يحدد حجم مجموعة الاختبار ويتم حسابه عن طريق طرح قيمة `1 - train\_size`، وهذا يعني أن حجم مجموعة الاختبار سيكون 20% من البيانات الأصلية. المعامل `shuffle` محدد بـ `False` لضمان عدم إعادة ترتيب البيانات أثناء التقسيم.

ثالثاً عرض أشكال مجموعة التدريب والاختبار:

استخدام الدالة `print` لعرض أشكال مجموعة التدريب ومجموعة الاختبار. يستخدم `train\_data.shape` لعرض عدد الصفوف والأعمدة في مجموعة التدريب، ويستخدم `test\_data.shape` لعرض عدد الصفوف والأعمدة في مجموعة الاختبار.

النتائج:

```
# Display the shapes of training and testing sets
print("Training set shape:", train_data.shape)
print("Testing set shape:", test_data.shape)
```



```
[*****100%*****] 1 of 1 completed
```

Date	Open	High	Low	Close	Adj Close	\
2020-01-02	1518.099976	1528.699951	1518.000000	1524.500000	1524.500000	
2020-01-03	1530.099976	1552.699951	1530.099976	1549.199951	1549.199951	
2020-01-06	1580.000000	1580.000000	1560.400024	1566.199951	1566.199951	
2020-01-07	1558.300049	1576.300049	1558.300049	1571.800049	1571.800049	
2020-01-08	1579.699951	1604.199951	1552.300049	1557.400024	1557.400024	

Volume

Date	Volume
2020-01-02	214
2020-01-03	107
2020-01-06	416
2020-01-07	47
2020-01-08	236

timestamp	price
0 2020-01-02	1524.500000
1 2020-01-03	1549.199951
2 2020-01-06	1566.199951
3 2020-01-07	1571.800049
4 2020-01-08	1557.400024

Training set shape: (893, 2)
Testing set shape: (224, 2)

Task 2: LSTM Model Development

1. Import necessary libraries such as `numpy`, `pandas`, `tensorflow` (or `keras` with TensorFlow 2.x), and `yfinance`.

```
[ ] import numpy as np
import pandas as pd
import tensorflow as tf
import yfinance as yf
from sklearn.preprocessing import MinMaxScaler
```

NumPy هي مكتبة قوية للحسابات العلمية والعمليات الرياضية في Python. يتم استخدام عادة للتعامل مع البيانات المتعددة الأبعاد والعمليات الرياضية المتقدمة.

pandas هي مكتبة شائعة في Python لتحليل وتنظيم البيانات. توفر واجهات برمجة مريحة للعديد من العمليات المتعلقة بالبيانات مثل قراءة وكتابة الملفات، وتصفية وتجميع البيانات، وإجراء العمليات الإحصائية والتحويلات البيانية.

TensorFlow هي مكتبة مفتوحة المصدر تستخدم لبناء وتدريب نماذج الذكاء الاصطناعي وتنفيذ الحوسبة الرقمية العالية الأداء بشكل عام. تستخدم TensorFlow بشكل واسع في تطبيقات تعلم الآلة والتعلم العميق.

yfinance هي مكتبة تسمح بالوصول إلى بيانات الأسهم والعملات من Yahoo Finance بسهولة. توفر واجهة برمجة التطبيقات (API) لاستعلام واسترجاع البيانات التاريخية والمعلومات المالية الأخرى.

from sklearn.preprocessing import MinMaxScaler
تستخدم هذه السطر لاستدعاء الدالة `MinMaxScaler` من مكتبة sklearn.preprocessing. MinMaxScaler هي واحدة من وظائف التحويل المتوفرة في مكتبة (sklearn) scikit-learn، والتي تستخدم لتحويل البيانات إلى مقياس محدد، عادةً من 0 إلى 1. يستخدم هذا التحويل لتطبيق القيود على نطاق البيانات وتحسين أداء بعض النماذج الإحصائية والتعلم الآلي.

2. Construct an LSTM model architecture for time series prediction.

```
# Set the sequence length
import numpy as np
import pandas as pd
from sklearn.preprocessing import MinMaxScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dropout, Dense

seq_length=60
model = Sequential()
model.add(LSTM(units=50, return_sequences=True, input_shape=(seq_length, 1)))
model.add(Dropout(0.2))
model.add(LSTM(units=50, return_sequences=True))
model.add(Dropout(0.2))
model.add(LSTM(units=50))
model.add(Dropout(0.2))
model.add(Dense(units=1))
model.compile(optimizer='adam', loss='mean_squared_error')
```

بناء نموذج شبكة عصبية تستخدم LSTM (Long Short-Term Memory) للتنبؤ بالبيانات الزمنية.

الخطوات:

يتم تعيين المتغير `seq\_length` بقيمة 60. هذا المتغير يحدد طول المتسلسلة (sequence length)، وهو عدد الفترات الزمنية التي ستستخدم للتنبؤ بالفترة الزمنية التالية.

تم إنشاء كائن Sequential، وهو نموذج فارغ يمكننا إضافة الطبقات إليه بتسلسل.

يتم إضافة طبقة LSTM إلى النموذج. هذه الطبقة تحتوي على وحدات LSTM بعدد 50 وتستخدم `return\_sequences=True` لإرجاع تسلسل النتائج بدلاً من إرجاع قيمة واحدة. المعامل `input\_shape` يحدد شكل البيانات الواردة إلى الطبقة، حيث يكون لدينا مدخلات بشكل (seq_length, 1)، حيث seq_length هو الطول المحدد مسبقاً.

يتم إضافة طبقة Dropout بنسبة 0.2 بعد طبقة LSTM الأولى. Dropout هي تقنية تستخدم لتجنب التشبع وتحسين قدرة النموذج على العمومية.

يتم إضافة طبقة LSTM ثانية مع وحدات LSTM بعدد 50 وتستخدم
`return\_sequences=True` لإرجاع تسلسل النتائج.

يتم إضافة طبقة Dropout بنسبة 0.2 بعد الطبقة LSTM الثانية.

يتم إضافة طبقة LSTM ثالثة مع وحدات LSTM بعدد 50. في هذه
الحالة، نحدد `return\_sequences=False` بشكل افتراضي، وبالتالي لا
يتم إرجاع تسلسل النتائج.

يتم إضافة طبقة Dropout بنسبة 0.2 بعد الطبقة LSTM الثالثة.

يتم إضافة طبقة Dense بوحدّة واحدة. هذه الطبقة تستخدم لتوليد
النتائج النهائية.

تتم عملية تجميع النموذج بواسطة تحديد المحسن (optimizer) ودالة
الخسارة (loss) في هذه الحالة، يتم استخدام المحسن "adam" ودالة
الخسارة "mean_squared_error" التي تستخدم لمشكلة التنبؤ
بالبيانات الرقمية.

3. Normalize the gold price data to bring it within a specific range for better convergence during training.

```
[ ] # Define the desired range for scaling
    desired_range = (0, 1)

    # Create a MinMaxScaler object and scale the training data
    scaler = MinMaxScaler(feature_range=desired_range)
    train_scaled = scaler.fit_transform(train_data[['price']])

    # Scale the test data using the same scaler
    test_scaled = scaler.transform(test_data[['price']])
```

إنشاء المتغير `desired\_range` لتحديد النطاق المطلوب للتدريج (scaling) في هذه الحالة، يتم تعيين النطاق المطلوب بين 0 و 1.

إنشاء كائن MinMaxScaler وتعيينه إلى المتغير `scaler` يتم تمرير `feature\_range` كوسيلة لتحديد النطاق المرغوب للتدريج. في هذه الحالة، يتم استخدام النطاق المحدد في المتغير `desired\_range` (0 و 1)

استخدام المرحلة المتدرجة على بيانات التدريب. يتم تطبيق الدالة `fit\_transform` على عمود "price" في `train\_data` تقوم هذه الدالة بتدريج البيانات بناءً على النطاق المحدد في `scaler` وتعيد البيانات المدرجة. يتم تخزين النتائج في `train\_scaled`

استخدام المدرج على بيانات الاختبار. يتم تطبيق الدالة `transform` على عمود "price" في `test\_data` باستخدام نفس `scaler` الذي تم استخدامه لتدريج بيانات التدريب. تقوم هذه الدالة بتدريج البيانات بناءً على النطاق المحدد سابقًا وتعيد البيانات المدرجة. يتم تخزين النتائج في `test\_scaled`

4. Train the LSTM model using the training data, experimenting with different hyperparameters.

```
[ ] # Prepare the training data
X_train = []
y_train = []

# Generate training sequences
for i in range(seq_length, len(train_scaled)):
    X_train.append(train_scaled[i-seq_length:i, 0])
    y_train.append(train_scaled[i, 0])

# Convert the lists to numpy arrays
X_train, y_train = np.array(X_train), np.array(y_train)

# Reshape the input data to fit the LSTM model
X_train = np.reshape(X_train, (X_train.shape[0], X_train.shape[1], 1))

# Define the input shape for the LSTM layer
lstm_input_shape = (X_train.shape[1], 1)
```

إنشاء مجموعات البيانات المستخدمة في تدريب النموذج LSTM وتهيئة بيانات الدخل لتناسب النموذج. الخطوات:

إنشاء قوائم فارغة X_train و y_train لتخزين مجموعات البيانات التدريب. ثم استخدام حلقة لتوليد مجموعات البيانات التدريب. تبدأ الحلقة من seq_length وتستمر حتى نهاية بيانات التدريب بعد التدريب.

وإضافة مجموعة البيانات التدريب الحالية إلى قوائم X_train و y_train يتم استخراج التسلسل الزمني للمدخلات من i-seq_length إلى i والقيمة التالية المتوقعة في y_train من train_scaled. ثم يتم تحويل قوائم X_train و y_train إلى مصفوفات NumPy

بعدها يتم إعادة تشكيل بيانات الدخل X_train لتناسب مع شكل البيانات المقبولة بواسطة طبقات LSTM. الشكل الجديد هو (عدد العينات، طول التسلسل، 1)، حيث يتم إضافة البعد الجديد (1) للإشارة إلى القيمة الواحدة المستخدمة في التنبؤ.

وأخيرا تعريف المتغير lstm_input_shape لتحديد شكل البيانات المتوقعة كمدخل لطبقة LSTM الشكل هو (طول التسلسل، 1)، حيث يتم تحديد طول التسلسل والبعد (1) للإشارة إلى القيمة الواحدة المستخدمة في التنبؤ.

Task 3: Model Evaluation and Prediction

1. Generate predictions for the testing set using the trained LSTM model.

```
# Prepare the training data
X_test = []
y_test = []
# Generate training sequences
for i in range(seq_length, len(test_scaled)):
    X_test.append(test_scaled[i-seq_length:i, 0])
    y_test.append(test_scaled[i, 0])
# Convert the lists to numpy arrays
X_test, y_test = np.array(X_test), np.array(y_test)
# Reshape the input data to fit the LSTM model
X_test = np.reshape(X_test, (X_test.shape[0], X_test.shape[1], 1))
history = model.fit(X_train, y_train, epochs=12, batch_size=32, validation_data=(X_test, y_test), verbose=1)
```

```
Epoch 1/12
27/27 [=====] - 11s 155ms/step - loss: 0.0610 - val_loss: 0.0636
Epoch 2/12
27/27 [=====] - 4s 153ms/step - loss: 0.0132 - val_loss: 0.0984
Epoch 3/12
27/27 [=====] - 3s 99ms/step - loss: 0.0113 - val_loss: 0.0678
Epoch 4/12
27/27 [=====] - 3s 99ms/step - loss: 0.0093 - val_loss: 0.0438
Epoch 5/12
27/27 [=====] - 2s 91ms/step - loss: 0.0094 - val_loss: 0.0457
Epoch 6/12
27/27 [=====] - 3s 102ms/step - loss: 0.0095 - val_loss: 0.0076
Epoch 7/12
27/27 [=====] - 4s 160ms/step - loss: 0.0096 - val_loss: 0.0461
Epoch 8/12
27/27 [=====] - 2s 93ms/step - loss: 0.0085 - val_loss: 0.0171
Epoch 9/12
27/27 [=====] - 2s 90ms/step - loss: 0.0076 - val_loss: 0.0597
Epoch 10/12
27/27 [=====] - 3s 93ms/step - loss: 0.0117 - val_loss: 0.0569
Epoch 11/12
27/27 [=====] - 3s 103ms/step - loss: 0.0088 - val_loss: 0.0309
Epoch 12/12
27/27 [=====] - 4s 150ms/step - loss: 0.0072 - val_loss: 0.0192
```

ستعد بيانات الاختبار ويقوم بتدريب النموذج LSTM باستخدام بيانات التدريب والاختبار الخطوات:

إنشاء قوائم فارغة X_test و y_test لتخزين مجموعات البيانات الاختبار

ثم تقوم باستخدام حلقة لتوليد مجموعات البيانات الاختبار. تبدأ الحلقة من seq_length وتستمر حتى نهاية بيانات الاختبار بعد التدريب

بعدها إضافة مجموعة البيانات الاختبار الحالية إلى قوائم `X_test` و `y_test`. يتم استخراج التسلسل الزمني للمدخلات من `i-seq_length` إلى `i` والقيمة التالية المتوقعة في `i` من `test_scaled`

ثم تحويل قوائم `X_test` و `y_test` إلى مصفوفات NumPy.

من ثم إعادة تشكيل بيانات الدخل `X_test` لتناسب مع شكل البيانات المقبولة بواسطة طبقات LSTM الشكل الجديد هو (عدد العينات، طول التسلسل، 1)، حيث يتم إضافة البعد الجديد (1) للإشارة إلى القيمة الواحدة المستخدمة في التنبؤ.

أخيرا تدريب النموذج باستخدام بيانات التدريب `X_test` و `y_test`. يتم تحديد عدد الحقول (epochs) وحجم الدُفعات (batch_size) وبيانات التحقق (validation_data) التي يتم استخدامها للتقييم يتم تخزين تاريخ التدريب والتحقق في المتغير `history`

2. Evaluate the model's performance using metrics like Mean Squared Error (MSE) or Root Mean Squared Error (RMSE).

```
[ ] # Make predictions using the trained model
predictions = model.predict(X_test)
predictions = scaler.inverse_transform(predictions)
# Invert the scaling to get actual prices
y_test_actual = scaler.inverse_transform(np.array(y_test).reshape(-1, 1))
```

6/6 [=====] - 1s 27ms/step

```
mse=np.mean(predictions - y_test)**2
rmse = np.sqrt(mse)

print("(MSE):", mse)
print("(RMSE):", rmse)
```

(MSE): 4252416.039242932
(RMSE): 2062.1387051415654

يستخدم النموذج المدرب لإجراء توقعات ويقوم بحساب مقاييس الأداء لتقييم النموذج

الخطوات:

اولا استخدام النموذج المدرب لتوقع القيم المستهدفة باستخدام بيانات الاختبار X_{test} يتم تخزين التوقعات في المتغير $predictions$

ثم استخدام $scaler$ لعكس التحويل السابق واستعادة قيم التوقعات الأصلية قبل التحويل.

بعدها استخدام $scaler$ أيضًا لاستعادة القيم الأصلية للبيانات الفعلية للاختبار (y_{test})

ثم نقوم بحساب متوسط مربع الخطأ (Mean Squared Error) بين التوقعات والقيم الفعلية للاختبار.

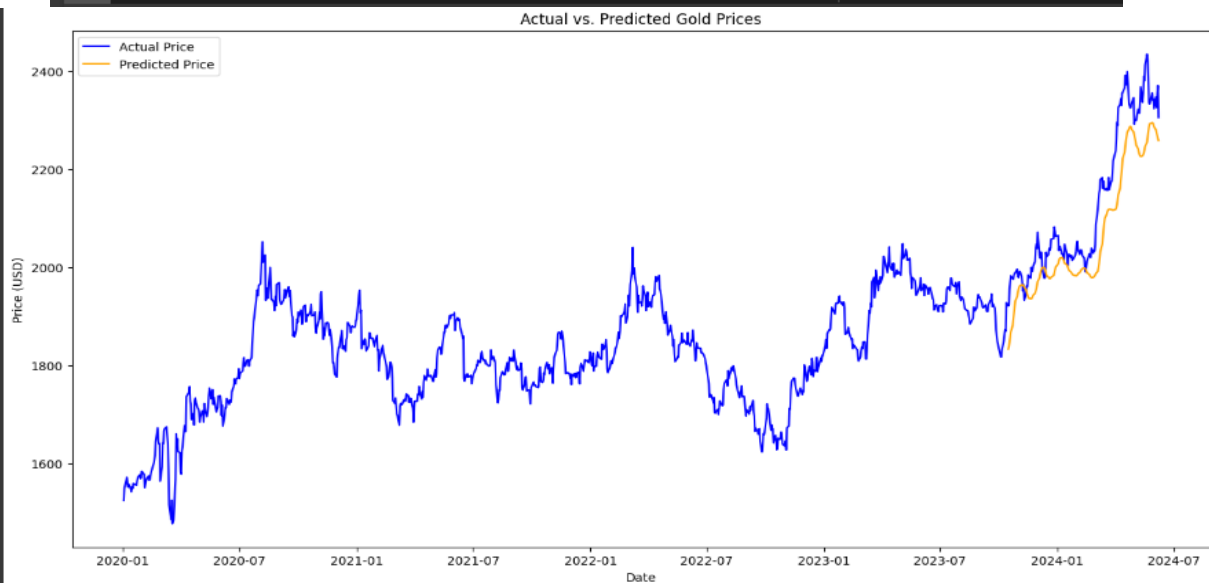
ايضا حساب الانحراف المعياري للمربعات الجذرية (Root Mean Squared Error) باستخدام قيمة متوسط مربع الخطأ.

تُطبع قيمة متوسط مربع الخطأ (MSE) وقيمة الانحراف المعياري للمربعات الجذرية (RMSE) لتقييم أداء النموذج.

3. Visualize the predicted gold prices alongside the actual gold prices using libraries like `matplotlib`.

```
import matplotlib.pyplot as plt

# Plotting actual vs. predicted prices
plt.figure(figsize=(16, 8))
plt.plot(gold_data['timestamp'], gold_data['price'], color='blue', label='Actual Price')
plt.plot(test_data['timestamp'].values[seq_length:], predictions, color='orange', label='Predicted Price')
plt.title('Actual vs. Predicted Gold Prices')
plt.xlabel('Date')
plt.ylabel('Price (USD)')
plt.legend()
plt.show()
```



```
import matplotlib.pyplot as plt

plt.figure(figsize=(16, 8))
plt.plot(test_data.index[-len(predictions):], y_test_actual, color='orange')
plt.plot(test_data.index[-len(predictions):], predictions, color='blue')
plt.title('Price Prediction')
plt.xlabel('Price (USD)')
plt.ylabel('Date')
plt.legend(['Predictions', 'Prices'])
plt.show()
```



Task 4: Visualization Enhancement and Analysis

1. Create more advanced visualizations, such as comparing actual vs. predicted prices over a longer time horizon.

```
# Prepare input data for LSTM on the full dataset
all_data_scaled = scaler.fit_transform(np.array(gold_data['price']).reshape(-1, 1))
X_all, y_all = [], []
for i in range(seq_length, len(all_data_scaled)):
    X_all.append(all_data_scaled[i-seq_length:i, 0])
    y_all.append(all_data_scaled[i, 0])

X_all, y_full = np.array(X_all), np.array(y_all)

# Reshape input data for LSTM
X_all = np.reshape(X_all, (X_all.shape[0], X_all.shape[1], 1))

# Make predictions using the trained model on the full dataset
all_predictions = model.predict(X_all)

# Invert the scaling to get actual prices
all_predictions = scaler.inverse_transform(all_predictions)
y_all_actual = scaler.inverse_transform(np.array(y_full).reshape(-1, 1))

# Plot actual vs. predicted prices over a longer time horizon
plt.figure(figsize=(16, 8))
plt.plot(gold_data.index[-len(all_predictions):], y_all_actual, label='Actual Prices', color='blue')
plt.plot(gold_data.index[-len(all_predictions):], all_predictions, label='Predicted Prices', color='red')
plt.title('Actual vs. predicted prices over a longer time horizon')
plt.xlabel('Gold Price (USD)')
plt.ylabel('Date')
plt.legend()
plt.show()
```

تحضير البيانات الكاملة لاستخدامها في نموذج LSTM، ثم يتم استخدام النموذج لتوقع الأسعار على مدى طويل من الوقت. يتم استعادة القيم الأصلية للأسعار ويتم رسم الأسعار الفعلية والمتوقعة للتحقق من أداء النموذج

الخطوات:

يتم استخدام scaler لتحويل جميع البيانات (الأسعار) إلى نطاق معين، مع تحويلها إلى مصفوفة بشكل مناسب.

تُنشئ قوائم فارغة X_all و y_all لتخزين مجموعات البيانات.

يتم استخدام حلقة لإنشاء مجموعات البيانات. تبدأ الحلقة من seq_length وتستمر حتى نهاية البيانات المحولة بعد التدرج.

يتم إضافة مجموعة البيانات الحالية إلى قوائم `X_all` و `y_all` يتم استخراج التسلسل الزمني للمدخلات من `i-seq_length` إلى `i` والقيمة التالية المتوقعة في `i` من `all_data_scaled`

تحويل قوائم `X_all` و `y_all` إلى مصفوفات NumPy.

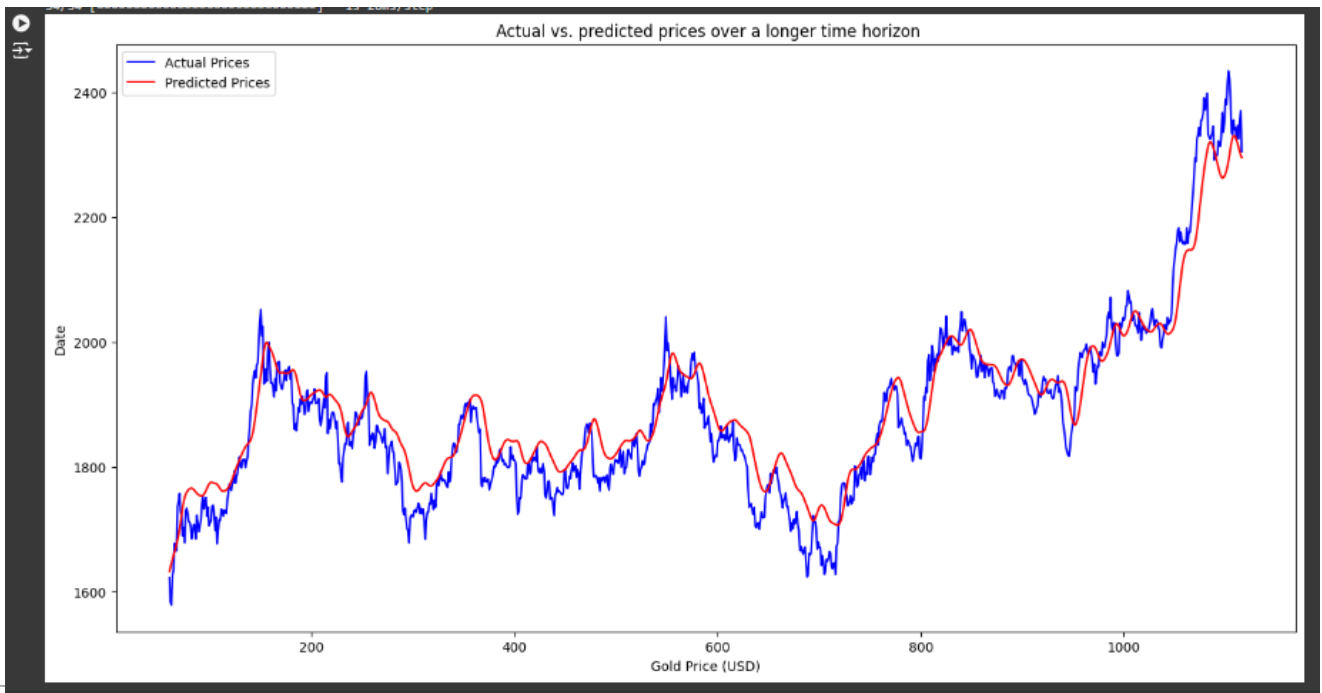
إعادة تشكيل بيانات الدخل `X_all` لتناسب مع شكل البيانات المقبولة بواسطة طبقات LSTM الشكل الجديد هو (عدد العينات، طول التسلسل، 1)، حيث يتم إضافة البعد الجديد (1) للإشارة إلى القيمة الواحدة المستخدمة في التنبؤ.

يتم استخدام النموذج المدرب لتوقع الأسعار باستخدام البيانات الكاملة `X_all` يتم تخزين التوقعات في المتغير `all_predictions`

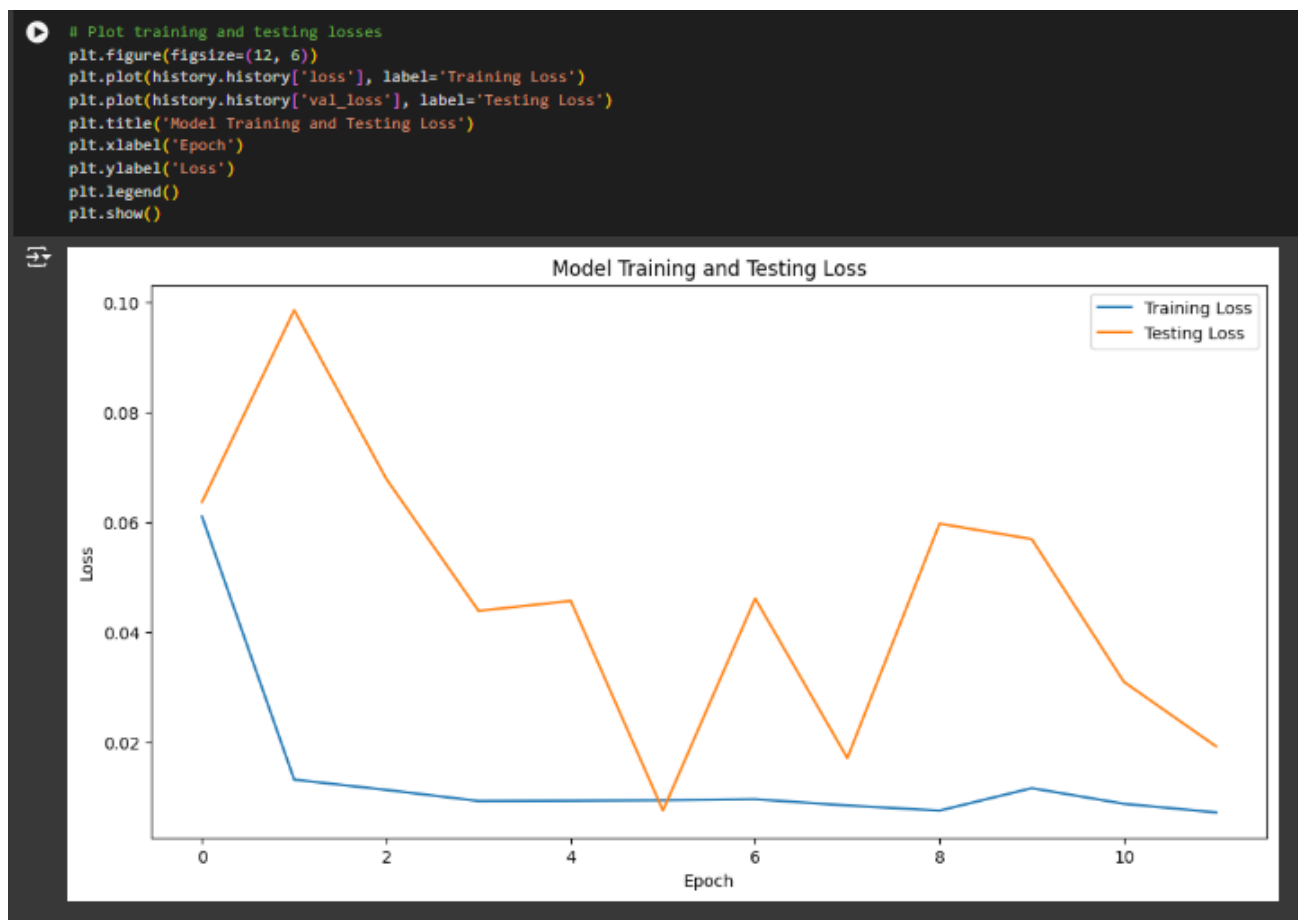
استخدام `scaler` لعكس التحويل السابق واستعادة القيم الأصلية للتوقعات قبل التحويل.

يتم استخدام `scaler` أيضًا لاستعادة القيم الأصلية للبيانات الفعلية `y_full` للأسعار.

يتم رسم الأسعار الفعلية والمتوقعة على مدى طويل من الوقت باستخدام المكتبة الرسومية Matplotlib يتم تحديد عنوان الرسم البياني وتسمية المحاور وعرض الأسعار الفعلية باللون الأزرق والأسعار المتوقعة باللون الأحمر. تُطبع الرسمة للعرض.



2. Plot the training and testing losses during model training to understand convergence.



رسم منحنيات الخسارة (Loss) لعملية التدريب والاختبار للنموذج الخطوات:

يتم رسم منحنى الخسارة (Loss) لعملية التدريب. يتم استخدام `history.history['loss']` للوصول إلى قائمة الخسارة التي تم تسجيلها خلال كل دورة تدريب.

يتم رسم منحنى الخسارة (Loss) لعملية الاختبار. يتم استخدام `history.history['val_loss']` للوصول إلى قائمة الخسارة التي تم تسجيلها خلال كل دورة اختبار.

يتم تعيين عنوان للرسم البياني يوضح أنه يتعلق بخسارة التدريب والاختبار للنموذج.

تُضاف تسميات لمحاور الرسم البياني، حيث يكون المحور الأفقي يُعبر عن الحقب (Epoch) والمحور العمودي يُعبر عن الخسارة.

3. Analyze the accuracy of the predictions and discuss the potential impact of varying model hyperparameters.

```
[ ] # Task 3: Analyzing prediction accuracy
mse = np.mean((predictions - y_test_actual)**2)
rmse = np.sqrt(mse)
mae = np.mean(np.abs(predictions - y_test_actual))
correlation = np.corrcoef(predictions.flatten(), y_test_actual.flatten())[0, 1]
print("Mean Squared Error (MSE):", mse)
print("Root Mean Squared Error (RMSE):", rmse)
print("Mean Absolute Error (MAE):", mae)
print("Correlation Coefficient:", correlation)
```



```
Mean Squared Error (MSE): 6338.470339117979
Root Mean Squared Error (RMSE): 79.61451085774489
Mean Absolute Error (MAE): 66.1399833865282
Correlation Coefficient: 0.9579282241568603
```

تحليل الخرج:

Mean Squared Error (MSE): 6338.470339117979 يشير إلى المتوسط المربع للفرق بين التوقعات والقيم الفعلية. قيمة عالية تشير إلى ارتفاع الانحراف بين التوقعات والقيم الفعلية.

Root Mean Squared Error (RMSE): 79.61451085774489 يعبر عن جذر المتوسط المربع للفرق بين التوقعات والقيم الفعلية. يُقدر بنفس وحدة القياس كما هي القيم الفعلية. قيمة أقل تشير إلى دقة أفضل للتوقعات.

Mean Absolute Error (MAE): 66.1399833865282 يعبر عن المتوسط المطلق للفرق بين التوقعات والقيم الفعلية. يقيس الانحراف المطلق للتوقعات عن القيم الفعلية. قيمة أقل تشير إلى دقة أفضل للتوقعات.

Correlation Coefficient: 0.957928224156860 يعبر عن معامل الارتباط بين التوقعات والقيم الفعلية. قيمة تقترب من 1 تشير إلى وجود ارتباط قوي وموجب بين التوقعات والقيم الفعلية، بينما قيمة تقترب من -1 تشير إلى وجود ارتباط قوي وسالب بينهما. في هذه الحالة، قيمة 0.9579282241568603 تشير إلى وجود ارتباط قوي وموجب بين التوقعات والقيم الفعلية.

Task 5: Fine-Tuning and Comparison

1. Experiment with different LSTM model configurations, adjusting the number of hidden layers, LSTM units, dropout rates, and learning rates.

```
# Function to create and train LSTM model
def create_lstm_model(input_shape, lstm_units=64, dropout_rate=0.1):
    model = Sequential()
    model.add(LSTM(units=lstm_units, return_sequences=True, input_shape=input_shape))
    model.add(Dropout(dropout_rate))
    model.add(LSTM(units=lstm_units, return_sequences=True))
    model.add(Dropout(dropout_rate))
    model.add(LSTM(units=lstm_units))
    model.add(Dropout(dropout_rate))
    model.add(Dense(units=1))
    model.compile(optimizer='adam', loss='mean_squared_error')
    return model
```

انشاء دالة لتقوم بتمرير المعاملات لو اردنا ان نغيرها

```
# Normalize the data
gold_data['scaled_price'] = scaler.fit_transform(np.array(gold_data['price']).reshape(-1, 1))

# Prepare input data for LSTM
X_Comparison, y_Comparison = [], []
for i in range(60, len(gold_data)):
    X_Comparison.append(gold_data['scaled_price'][i-60:i])
    y_Comparison.append(gold_data['scaled_price'][i])

X_Comparison, y_Comparison = np.array(X_Comparison), np.array(y_Comparison)

# Reshape input data for LSTM
X_Comparison = np.reshape(X_Comparison, (X_Comparison.shape[0], X_Comparison.shape[1], 1))
```

تقوم بتحضير البيانات لتدريب النموذج LSTM. يتم تطبيع البيانات لضمان توازنها وتسهيل عملية التدريب. ثم يتم تجهيز البيانات المعالجة والمستهدفة في شكل متوالي مناسب لدخول النموذج LSTM

```
# Split the data into training and testing sets
train_size = int(len(X_Comparison) * 0.8)
train_data, test_data, train_labels, test_labels = X_Comparison[:train_size], X_Comparison[train_size:], y_Comparison[:train_size], y_Comparison[train_size:]

# Experiment with different model configurations
configs = [
    {'lstm_units': 50, 'dropout_rate': 0.2},
    {'lstm_units': 100, 'dropout_rate': 0.3},
    {'lstm_units': 75, 'dropout_rate': 0.1}
]
```

حساب حجم مجموعة التدريب بناءً على نسبة مئوية محددة من حجم البيانات الكلي. في هذه الحالة، يتم استخدام 80٪ من البيانات للتدريب و20٪ للاختبار.

ثم يتم تقسيم بيانات التدريب والاختبار والبيانات المستهدفة إلى مجموعتين مستقلتين. تأخذ train_data و train_labels البيانات

والبيانات المستهدفة لمجموعة التدريب، بينما تأخذ test_data و test_labels البيانات والبيانات المستهدفة لمجموعة الاختبار. تستخدم الفهرسات [:train_size] و [:train_size] لتحديد النطاق المناسب من البيانات لكل مجموعة.

ويتم تعريف قائمة configs التي تحتوي على متغيرات مختلفة للنموذج. كل متغير يتم تمثيله كقاموس يحتوي على مفاتيح lstm_units و dropout_rate و قيمهما. يمكنك تجربة متغيرات مختلفة للنموذج من خلال تعديل هذه القائمة.

```
for config in configs:
    model = create_lstm_model((X_Comparison.shape[1], 1), lstm_units=config['lstm_units'], dropout_rate=config['dropout_rate'])
    history = model.fit(train_data, train_labels, epochs=3, batch_size=32, validation_data=(test_data, test_labels), verbose=0)

    # Evaluate the model
    predictions = model.predict(test_data)
    predictions = scaler.inverse_transform(predictions)
    y_actual = scaler.inverse_transform(np.array(test_labels).reshape(-1, 1))

    mse = mean_squared_error(y_actual, predictions)
    rmse = sqrt(mse)

    print(f"Configuration: {config}")
    print(f"Root Mean Squared Error (RMSE): {rmse}")
```

تدريب النموذج وتقييم أدائه باستخدام التكوينات المختلفة.
الخطوات:

اولا استدعاء كل تكوين من القائمة configs في كل تكرار للحلقة.

ثم إنشاء نموذج LSTM باستخدام create_lstm_model() وتزوير الأبعاد المناسبة للبيانات المدخلة وقيم التكوين من الحلقة الحالية.

ويتم تدريب النموذج باستخدام بيانات التدريب والبيانات المستهدفة المقابلة. يتم تحديد عدد الحقول التي يتم تدريبها (epochs) وحجم الدفعة (batch_size) والبيانات المستخدمة للتحقق من صحة النموذج (validation_data) ومستوى التفاصيل الذي يتم طباعته (verbose)

من بعدها استخدام النموذج للتنبؤ بالأسعار باستخدام بيانات الاختبار.

يتم استعادة القيم الأصلية للأسعار المتنبئ بها بعد عملية التنبؤ. يتم استخدام scaler.inverse_transform() لتحويل القيم المقاسة إلى القيم الأصلية

ثم استعادة القيم الأصلية للأسعار المستهدفة للمقارنة بالتوقعات. يتم تحويل test_labels إلى مصفوفة NumPy وإعادة تشكيلها لتكون ثنائية الأبعاد. ويقوم حساب خطأ المربعات المتوسط (MSE) بين القيم الفعلية والتوقعات. ويقوم حساب جذر خطأ المربعات المتوسط (RMSE) من قيمة MSE الخرج :

```
7/7 [=====] - 1s 23ms/step
Configuration: {'lstm_units': 50, 'dropout_rate': 0.2}
Root Mean Squared Error (RMSE): 96.00700089463312
7/7 [=====] - 2s 60ms/step
Configuration: {'lstm_units': 100, 'dropout_rate': 0.3}
Root Mean Squared Error (RMSE): 54.89025704843927
7/7 [=====] - 10s 52ms/step
Configuration: {'lstm_units': 75, 'dropout_rate': 0.1}
Root Mean Squared Error (RMSE): 73.21823262603615
```

2. Compare the performance of different models using appropriate evaluation metrics.

تحليل الخرج:

Root Mean Squared Error (RMSE): 96.00700089463312
قيمة RMSE للمتغيرات {lstm_units: 50, dropout_rate: 0.2} هي 96.00700089463312. يُستخدم RMSE لقياس مقدار الانحراف المتوسط بين القيم الفعلية والقيم المتنبأ. وبما أن القيمة هنا عالية، فإن النموذج قد يكون يعاني من أداء ضعيف في تنبؤ الأسعار.

Root Mean Squared Error (RMSE): 54.89025704843927
قيمة RMSE للمتغيرات {lstm_units: 100, dropout_rate: 0.3} هي 54.89025704843927. يكون RMSE أقل في هذه الحالة مقارنة بالمتغير السابق، مما يشير إلى أداء محسن للنموذج في تنبؤ الأسعار.

Root Mean Squared Error (RMSE): 73.21823262603615
قيمة RMSE للمتغيرات {lstm_units: 75, dropout_rate: 0.1} هي 73.21823262603615. هذا يعني أن النموذج يحقق أداءً متوسطًا في تنبؤ الأسعار، ولكن قد يكون هناك زيادة في الانحراف بين القيم الفعلية والقيم المتنبأ بالمقارنة مع المتغير الثاني.

3. Reflect on the insights gained from comparing various model setups.

توضح النتائج أن المتغير {lstm_units: 100, dropout_rate: 0.3} يحقق أداءً أفضل في تنبؤ الأسعار مقارنة بالمتغيرات الآخرين المستخدمين.

Task 6: Future Price Forecast

1. Extend the LSTM model to forecast gold prices beyond the existing dataset's timeframe.

```
# Prepare data for LSTM model
seq_length = 60
scaler = MinMaxScaler(feature_range=(0, 1))
gold_data['scaled_price'] = scaler.fit_transform(np.array(gold_data['price']).reshape(-1, 1))
X_future, y_future = [], []
for i in range(seq_length, len(gold_data)):
    X_future.append(gold_data['scaled_price'][i-seq_length:i])
    y_future.append(gold_data['scaled_price'][i])

X_future, y_future = np.array(X_future), np.array(y_future)
X_future = np.reshape(X_future, (X_future.shape[0], X_future.shape[1], 1))
train_size = int(len(X_future) * 0.8)
train_data, test_data, train_labels, test_labels = X_future[:train_size], X_future[train_size:], y[:train_size], y[train_size:]

if not isinstance(gold_data['timestamp'].iloc[0], pd.Timestamp):
    gold_data['timestamp'] = pd.to_datetime(gold_data['timestamp'])

# Create the LSTM model
model1 = create_lstm_model((X_future.shape[1], 1), lstm_units=75, dropout_rate=0.2)

# Train the model
history = model1.fit(train_data, train_labels, epochs=10, batch_size=32, validation_data=(test_data, test_labels), verbose=0)
```

قمنا بإضافة سطر جديد عن ما سبق وهو

```
if not isinstance(gold_data['timestamp'].iloc[0], pd.Timestamp):
    gold_data['timestamp'] =
pd.to_datetime(gold_data['timestamp'])
```

وهو يعمل على تحقق من نوع البيانات في عمود التوقيت (timestamp) في إطار بيانات الذهب (gold_data) إذا لم يكن نوع البيانات هو pd.Timestamp، فإنه يقوم بتحويل القيم إلى نوع pd.Timestamp باستخدام pd.to_datetime()

```
1/1 [=====] - 1s 1s/step
1/1 [=====] - 0s 51ms/step
1/1 [=====] - 0s 49ms/step
1/1 [=====] - 0s 49ms/step
1/1 [=====] - 0s 48ms/step
1/1 [=====] - 0s 48ms/step
1/1 [=====] - 0s 43ms/step
1/1 [=====] - 0s 45ms/step
1/1 [=====] - 0s 42ms/step
1/1 [=====] - 0s 47ms/step
1/1 [=====] - 0s 50ms/step
1/1 [=====] - 0s 41ms/step
1/1 [=====] - 0s 45ms/step
1/1 [=====] - 0s 38ms/step
1/1 [=====] - 0s 54ms/step
1/1 [=====] - 0s 51ms/step
1/1 [=====] - 0s 51ms/step
1/1 [=====] - 0s 48ms/step
1/1 [=====] - 0s 53ms/step
1/1 [=====] - 0s 49ms/step
1/1 [=====] - 0s 50ms/step
1/1 [=====] - 0s 48ms/step
1/1 [=====] - 0s 51ms/step
1/1 [=====] - 0s 51ms/step
1/1 [=====] - 0s 52ms/step
1/1 [=====] - 0s 50ms/step
1/1 [=====] - 0s 44ms/step
1/1 [=====] - 0s 49ms/step
1/1 [=====] - 0s 50ms/step
1/1 [=====] - 0s 55ms/step
```

2. Utilize the model to predict gold prices for a defined number of future days.

```
# Make predictions for the future timeframe
future_data = gold_data['scaled_price'][-seq_length:]
future_predictions = []
num_predictions = 30 # Number of days to forecast
for _ in range(num_predictions):
    prediction = model.predict(np.array([future_data]))
    future_predictions.append(prediction[0, 0])
    future_data = np.append(future_data[1:], prediction[0])

# Inverse transform the predicted prices to the original scale
future_predictions = scaler.inverse_transform(np.array(future_predictions).reshape(-1, 1))

# Generate timestamps for the forecasted prices
last_timestamp = gold_data['timestamp'].iloc[-1]
timestamps = pd.date_range(start=last_timestamp, periods=num_predictions+1, freq='D')[1:]

# Create a DataFrame for the forecasted prices
forecast_df = pd.DataFrame({'timestamp': timestamps, 'price': future_predictions.flatten()})

# Concatenate the forecasted prices with the original dataset
gold_data = pd.concat([gold_data, forecast_df], ignore_index=True)
```

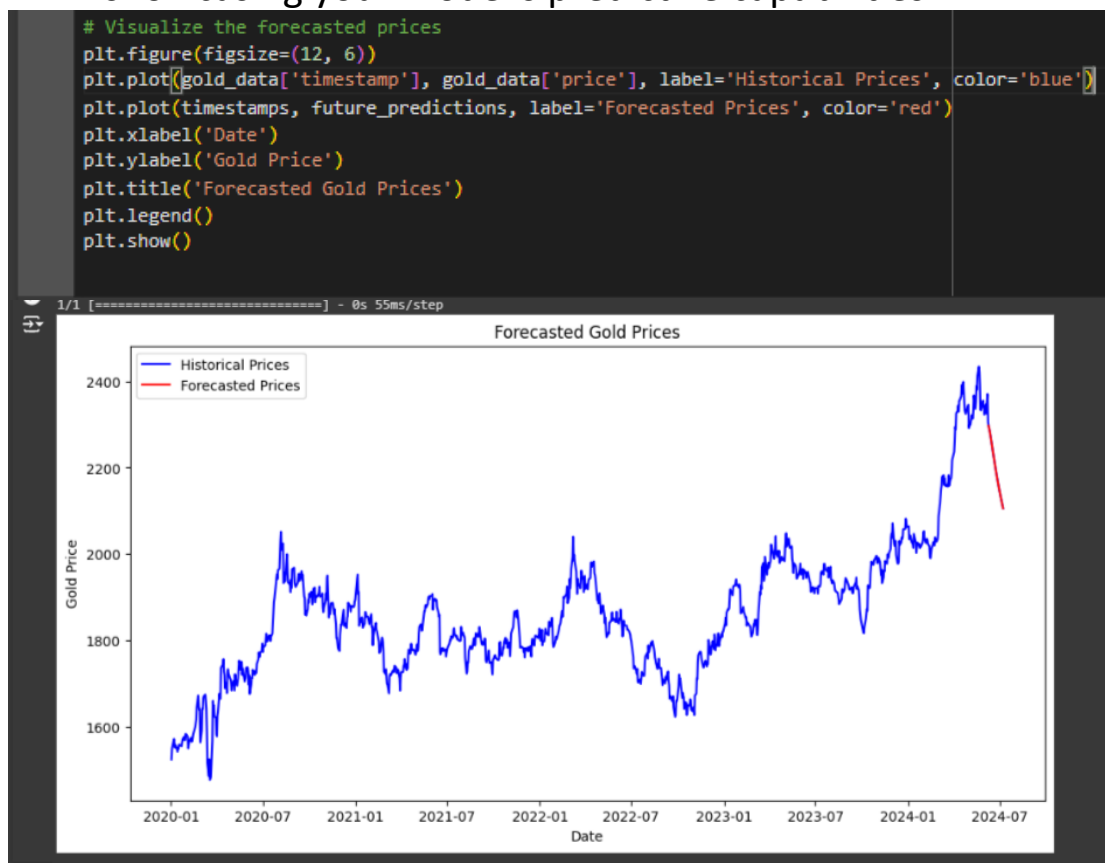
شرح الخطوات:

استخراج أحدث سلسلة من البيانات المحولة (scaled_price) من إطار بيانات الذهب (gold_data) بطول seq_length هذه السلسلة ستستخدم كبيانات الإدخال لتوليد التوقعات المستقبلية. إنشاء قائمة فارغة لتخزين التوقعات المستقبلية. تحديد عدد الأيام التي سيتم توقعها. بدء حلقة تكرارية لتوليد التوقعات المستقبلية لعدد الأيام المحدد. توليد توقع باستخدام النموذج LSTM يتم إدخال بيانات المستقبل (future_data) كإدخال للنموذج واستخراج التوقع الناتج. إضافة التوقع الحالي إلى قائمة التوقعات المستقبلية. تحديث بيانات المستقبل (future_data) ليتم استخدامها في التوقعات المستقبلية القادمة. يتم إزالة القيمة الأولى وإضافة التوقع الحالي في نهاية البيانات. تحويل التوقعات المستقبلية إلى مقياسها الأصلي باستخدام مقياس ال MinMaxScaler المعكوس (inverse_transform) يتم تغيير شكل المصفوفة إلى (-1, 1) قبل التحويل. استخراج الطابع الزمني (timestamp) من إطار بيانات الذهب.

إنشاء قائمة من الطوابع الزمنية المتوقعة للتوقعات المستقبلية. تبدأ القائمة من الطابع الزمني الأخير وتشمل عدد الأيام المحددة مع تردد يومي (freq='D')

إنشاء إطار بيانات (DataFrame) للتوقعات المستقبلية باستخدام الطوابع الزمنية والأسعار المتوقعة. يتم تسطيح (flatten) المصفوفة لتوقعات الأسعار لتتوافق مع شكل الإطار. دمج إطار بيانات التذهب الأصلي (gold_data) مع إطار بيانات التوقعات المستقبلية (forecast_df) باستخدام الدالة concat() من مكتبة pandas. يتم تجاهل الفهرس الأصلي للمصفوف باستخدام ignore_index=True لضمان ترقيم صفوف متساوٍ بعد الدمج.

3. Visualize both the historical and forecasted prices, showcasing your model's predictive capabilities.



Task 8: Creative Visualization

1. Implement creative visualizations using libraries like Plotly to showcase the volatility and trend patterns of gold prices.

```
import plotly.graph_objects as go

# Calculate the Simple Moving Average (SMA)
sma_periods = 20
gold_data['SMA'] = gold_data['price'].rolling(window=sma_periods).mean()

# Create a line chart for gold prices and SMA
line_chart = go.Scatter(
    x=gold_data['timestamp'],
    y=gold_data['price'],
    mode='lines',
    name='Gold Prices'
)

sma_chart = go.Scatter(
    x=gold_data['timestamp'],
    y=gold_data['SMA'],
    mode='lines',
    name='SMA ({}' .format(sma_periods)
)

# Create a layout for the chart
layout = go.Layout(
    title='Gold Prices with SMA',
    xaxis=dict(title='Date'),
    yaxis=dict(title='Price'),
    showlegend=True
)

# Combine the line chart, SMA chart, and layout in a single figure
fig = go.Figure(data=[line_chart, sma_chart], layout=layout)

# Show the figure
fig.show()
```



```

import plotly.express as px

# Calculate the daily price changes
gold_data['price_change'] = gold_data['price'].diff()

# Create a heatmap to visualize the stability of gold prices
heatmap = go.Heatmap(
    x=gold_data['timestamp'],
    y=gold_data['timestamp'],
    z=gold_data['price_change'].values.reshape(-1, 1),
    colorscale='RdYlGn', # Choose a colorscale
    colorbar=dict(title='Price Change')
)

# Create a layout for the heatmap
layout = go.Layout(
    title='Stability of Gold Prices',
    xaxis=dict(title='Date'),
    yaxis=dict(title='Date')
)

# Combine the heatmap and layout in a single figure
fig = go.Figure(data=heatmap, layout=layout)

# Show the figure
fig.show()

```



```

import plotly.graph_objects as go

# Create candlestick chart to visualize the trend patterns of gold prices
candlestick = go.Candlestick(
    x=gold_data['timestamp'],
    open=gold_data['price'],
    high=gold_data['price'],
    low=gold_data['price'],
    close=gold_data['price'],
    increasing=dict(line=dict(color='green')),
    decreasing=dict(line=dict(color='red'))
)

# Create a layout for the chart
layout = go.Layout(
    title='Trend Patterns of Gold Prices',
    xaxis=dict(title='Date'),
    yaxis=dict(title='Gold Price'),
    showlegend=False
)

# Combine the candlestick chart and layout in a single figure
fig = go.Figure(data=candlestick, layout=layout)

# Show the figure
fig.show()

```

